```python
import numpy as np
import matplotlib.pyplot as plt
def f_a(x):
    return np.cos(x)
def f_b(x):
    return 1 / (1 + 100*x**2)
def f_c(x):
    return np.sqrt(np.abs(x))
def trap(f, a, b, n):

    x = np.linspace(a, b, n)
    y = f(x)
    h = (b - a) / (n - 1)
    integral = h * (y[0] + y[-1]) / 2
    for i in range(1, n-1):
        integral += h * y[i]
    return integral
def simp(f, a, b, n):
    x = np.linspace(a, b, n)
    y = f(x)
    h = (b - a) / (n - 1)
    integral = y[0] + y[-1]
    for i in range(1, n-1, 2):
        integral += 4 * y[i]
    for i in range(2, n-1, 2):
        integral += 2 * y[i]
    return h * integral / 3
def adapt(f, a, b, tol=1e-6, record_points=False):
    eval_points = []
    def simp_step(a, b, fa, fm, fb):
        h = b - a
        return h * (fa + 4*fm + fb) / 6
    def recursive(a, b, fa, fb, S, tol, level=0):
        m = (a + b) / 2
```

```
        integral += 4 * y[i]
    for i in range(2, n-1, 2):
        integral += 2 * y[i]
    return h * integral / 3
def adapt(f, a, b, tol=1e-6, record_points=False):
    eval_points = []
    def simp_step(a, b, fa, fm, fb):
        h = b - a
        return h * (fa + 4*fm + fb) / 6
    def recursive(a, b, fa, fb, S, tol, level=0):
        m = (a + b) / 2
        fm = f(m)
        f_left = f((a + m) / 2)
        f_right = f((m + b) / 2)
        S1 = simp_step(a, m, fa, f_left, fm)
        S2 = simp_step(m, b, fm, f_right, fb)
        S_total = S1 + S2
        if record_points:
            eval_points.extend([a, m, b])
        if abs(S - S_total) <= 15 * tol:
            return S_total
        else:
            return recursive(a, m, fa, fm, S1, tol/2, level+1) + \
                   recursive(m, b, fm, fb, S2, tol/2, level+1)
    fa, fb = f(a), f(b)
    S = simp_step(a, b, fa, f((a + b) / 2), fb)
    result = recursive(a, b, fa, fb, S, tol)
    return result, sorted(set(eval_points))
def adapt_plot(f, a, b, tol, title):
    result, eval_points = adapt(f, a, b, tol, record_points=True)
    x = np.linspace(a, b, 1000)
    y = f(x)
    plt.plot(x, y, label='Function', color='black')
    y_points = [f(point) for point in eval_points]
```

```
    result = recursive(a, b, fa, fb, s, tol)
    return result, sorted(set(eval_points))
def adapt_plot(f, a, b, tol, title):
    result, eval_points = adapt(f, a, b, tol, record_points=True)
    x = np.linspace(a, b, 1000)
    y = f(x)
    plt.plot(x, y, label='Function', color='black')
    y_points = [f(point) for point in eval_points]
    plt.scatter(eval_points, y_points, color='red', s=10, label='Evalu
    plt.xlabel("x")
    plt.ylabel("f(x)")
    plt.title(title)
    plt.legend()
    plt.grid(True)
    plt.show()
a, b = -1, 1
print("(a):")
print(f"Trapezoidal rule: {trap(f_a, a, b, 100)}")
print(f"Simpson's rule: {simp(f_a, a, b, 100)}")
print(f"Adaptive Simpson's rule: {adapt(f_a, a, b)}")
print("\n(b):")
print(f"Trapezoidal rule: {trap(f_b, a, b, 100)}")
print(f"Simpson's rule: {simp(f_b, a, b, 100)}")
print(f"Adaptive Simpson's rule: {adapt(f_b, a, b)}")
print("\n(c):")
print(f"Trapez rule: {trap(f_c, a, b, 100)}")
print(f"Simpson rule: {simp(f_c, a, b, 100)}")
print(f"Adaptive rule: {adapt(f_c, a, b)}")
adapt_plot(f_a, a, b, tol=1e-4, title="cos(x)")
adapt_plot(f_b, a, b, tol=1e-7, title="1 / (1 + 100*x^2)")
adapt_plot(f_c, a, b, tol=1e-2, title="sqrt(|x|)")
```

```python
def f_a(x):
    return np.sqrt(x**3)

def f_b(x):
    return 1 / (1 + 10 * x**2)

def f_c(x):
    return (np.exp(-9 * x) + np.exp(-1024 * (x - 0.25)**2)) / np.sqrt(np.pi)

def f_d(x):
    return 50 / (np.pi * (2500 * x**2 + 1))

def f_e(x):
    return 1 / np.sqrt(np.abs(x))

def f_f(x):
    return 25 * np.exp(-25 * x)

def f_g(x):
    return np.log(x)


print("Results (Simpson's Rule):")
print(f"(a): {simp(f_a, 0, 1):.6f}")
print(f"(b): {simp(f_b, 0, 1):.6f}")
print(f"(c): {simp(f_c, 0, 1):.6f}")
print(f"(d): {simp(f_d, 0, 10):.6f}")
print(f"(e): {simp(f_e, -9, 100):.6f}")
print(f"(f): {simp(f_f, 0, 10):.6f}")
print(f"(g): {simp(f_g, 1e-10, 1):.6f}")
```

```python
import numpy as np



def simp(f, a, b, n=10000):
    if n % 2 == 1:
        n += 1
    x = np.linspace(a, b, n + 1)
    y = f(x)
    h = (b - a) / n

    integral = y[0] + y[-1]
    integral += 4 * np.sum(y[1:n:2])
    integral += 2 * np.sum(y[2:n-1:2])
    return integral * h / 3


def f_a(x):
    return np.sqrt(x**3)

def f_b(x):
    return 1 / (1 + 10 * x**2)

def f_c(x):
    return (np.exp(-9 * x) + np.exp(-1024 * (x - 0.25)**2)) / np.sqrt(np.pi)

def f_d(x):
    return 50 / (np.pi * (2500 * x**2 + 1))

def f_e(x):
    return 1 / np.sqrt(np.abs(x))

def f_f(x):
    return 25 * np.exp(-25 * x)
```